

Functions in JS

Lec-6

What are Functions ?

- Functions are **reusable blocks of code** designed to **perform a particular task**.
- Functions are only executed when they're **called** or **invoked**.
- Functions often **return a optional value** back to the caller.
- Whenever we use a **return** statement in a function, **code after it does not get executed**.

Why are functions used ?

- They help us follow **D.R.Y (Do not Repeat Your code)** principle and helps us **reuse our code**.
- They help us **organize** our code into **smaller parts**.
- They makes code **easier to read** and **maintain**.
- They help us **use same code** to **produce different results at different times**.

Declaring and calling a Function

- A function can be created or declared using a **function** keyword, **function name** and a **paranthesis ({ })**.
- In order to **run / execute** the function **we have to call it** using **function name** followed by **open close round brackets ()**.

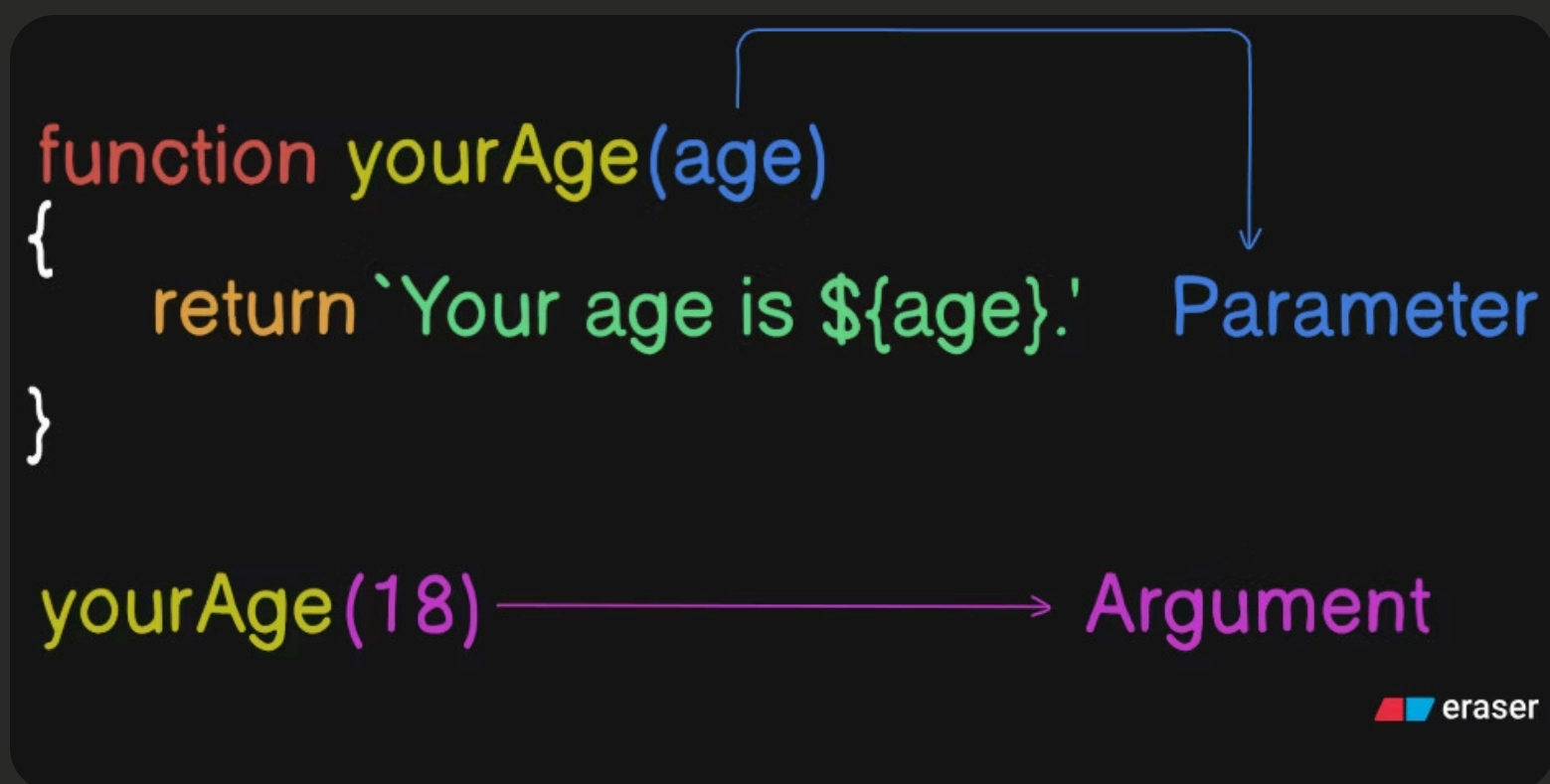
```
// function declaration
function sayMyName(){
  return `Hisenberg`;
}
// function calling
const call = sayMyName();
console.log(call) // will return 'Hisenberg'.
```



- Suppose, we have just written **sayMyName** instead of **sayMyName()** this meant we would've **just referenced a function** and **not have called it**.
1. **sayMyName** : refers to **function itself**, it **returns the function**.
 2. **sayMyName()** : refers to **function result**, it **returns the result**.

Parameters and Arguments in Functions

- **Parameters** : Parameters are the **placeholders** in which values are passed inside the function.
- **Arguments** : Arguments are the **values** which are passed to those placeholders (parameters).



- **Eg :**

```
function iceCreamOrder(order){
  return `You've ordered ${order} ice cream !!`
}
const orderRes = iceCreamOrder('chocolate')
console.log(orderRes); // will print "You've ordered chocolate ice cream !!"
```

- Functions can have **more than one parameters**.
- Arguments are assigned to parameters **in the order as they appear**.
- **Eg :**

```
function foodOrder(order,extra){
  return `You have ordered ${order} with a extra ${extra}.`
}
const orderDetails = foodOrder('burger','slice of cheese')
console.log(orderDetails); // will print "You have ordered burger with a extra slice of cheese."
```

- Functions can also have a **default parameters**.
- **Eg :**

```
function currentAge(birthYear,currentYear = 2026){
  return `Your age is: ${currentYear - birthYear}`
}
```

```
const ageDetails = currentAge(2007)
console.log(ageDetails); // will print: "Your age is: 19"
```

🔗 Solve This Practice Question

You're building a small feature for a food delivery app. Write a function named **placeOrder** that takes **two parameters**, **foodItem** and **quantity**, and **returns** a string like **"Ordering 2 portions of Biryani"** when called with **"Biryani"** and **2**.

- Call the function with a **food item** and **quantity of your choice**
- Store the returned result in a variable named **myOrder**.

Scope in JS

- Scope determines the **visibility** or **accessibility** of variables.
- JavaScript have 3 types of scopes:

1. **Global Scope**
2. **Function Scope**
3. **Block Scope**

Global Scope

- Variables declared **globally (outside any block or function)** have global scope.
- **Global variables can be accessed from anywhere** in the JavaScript program.
- Variables like **let**, **var** and **const** have same **global scope** when declared outside a block.
- **Eg :**

```
let number1 = 20 // global variable
```

```
function additionOfNumber(number2){  
    return number1 + number2 // global variable number1 can be accesible in this block as well.  
}
```

```
console.log(additionOfNumber(40));
```

Function Scope

- All JavaScript functions have their own scope.
- Variables defined **inside** a function are **not accessible (visible) from outside the function**.
- Variables declared with **let**, **var** and **const** are **quite similar** when declared inside a function.
- **Eg :**

```
function randomFunction(){  
    let randomVariable = 'random'  
}
```

```
console.log(randomVariable); // will give Uncaught ReferenceError: randomVariable is not defined
```

Block Scope

- Block Scope was introduced to JavaScript in **ES6**.
- Variables declared with **let** and **const** have **block scope** meaning they're **accessible only within that block**.
- **Eg :**

```
if(true){  
    let a = 40  
    const b = 60  
    var c = 80  
}  
console.log(a); // Uncaught ReferenceError: a is not defined  
console.log(b); // Uncaught ReferenceError: b is not defined  
console.log(c); // 80
```

Scope inside a function

- Whenever we define a function inside a another function the **variables declared inside a scope of parent function** can be **accessible inside the scope of child function**.
- But the **variables declared inside the scope of child function cannot** be **accessible in the scope of parent function**.
- Eg :**

```
function func1()
{
  let var1 = 69

  function func2()
  {
    let var2 = 43
    console.log(var1);
    console.log(var1 + var2);
  }

  func2()
  console.log(var2); // ReferenceError: var2 is not defined at func1
}

func1()
console.log(var1); // ReferenceError: var1 is not defined
```

Anonymous Function

- Functions **without any name** are anonymous functions.
- They are created using **function expression**.
- The **variable** in which they're stored, **it's name is used to call a function**.
- Eg :**

```
const anonymousVar = function(){
  return `Hi, I am an Anonymous function`
}

console.log(anonymousVar()); // will print "Hi, I am an Anonymous function"
```

Hoisting in JS

- It is a behavior of an interpreter, where **it appears to move declarations of variable, functions and classes** to the **top of their scope** before the code is executed.

- Function declaration after defining.**

```
function aFunction(){
  console.log(`Hi, I am a function`);
}

aFunction() // will print 'Hi, I am a function'
```

- Function declaration before defining.**

```
aFunction() // will print 'Hi, I am a function'
function aFunction(){
  console.log(`Hi, I am a function`);
}
```

- Declaration after function expression**

```
const anotherFunction = function(){
  console.log(`Hi, I am another function`);
}

anotherFunction() // will print 'Hi, I am anoter function'
```

- Declaration before function expression**

```
anotherFunction() // will give 'Uncaught ReferenceError:
Cannot access 'anotherFunction' before initialization.'

const anotherFunction = function(){
  console.log(`Hi, I am another function`);
}
```

?

Solve This Practice Question

You're building a **simple booking system for a movie theatre**. Create a **anonymous function** and store it in a variable **bookSeat** that takes **one parameter** called **movieName**. Inside this function, create another **named function** named as **generateTicket** that returns a message like **"Ticket booked for Inception"**.

- Call **generateTicket** from within **bookSeat** and return its result.
- The message in **generateTicket** should use the **movieName parameter** passed to **bookSeat**
- Do not call **generateTicket** outside of **bookSeat**

Arrow Functions

- An arrow function uses "`=>`" symbol.
- An arrow function is always written as a **function expression**.
- Arrow functions **allow to use shorter syntax** for function expression.
- You can **skip the function keyword** while writing the arrow function.
- **Eg :**

Normal Function Expression	Arrow Function
<pre>const egFunc = function(){ console.log('Normal function example') }</pre>	<pre>const egFunc = () =>{ console.log('Arrow Function example') }</pre>

- Arrow functions **return value by default**, if the **function has one statement that returns a value**.
- Thus in this case **you can remove curly brackets and return keyword** from your syntax.
- **Eg :**

Normal Function Expression	Arrow Function
<pre>const greetFunc = function(){ return `Greetings of the day` }</pre>	<pre>const greetFunc = () => `Greetings of the day`</pre>

- If you have **parameters**, you pass them **inside the parenthesis**.
- If you have **only one parameter**, you can also **skip parenthesis as well**.
- **Eg :**

Eg-1	Eg-2
<pre>const birthdayFn = (name) => `Happy birthday \${name}.`</pre>	<pre>const birthdayFn = name => `Happy birthday \${name}.`</pre>

- If you have more than one parameters, **implicit return** can be performed in the following way using **"round brackets ()"**

Normal Function Expression	Arrow Function
<pre>const multTwo = (numb1, numb2) => { return numb1*numb2 }</pre>	<pre>const multTwo = (numb1, numb2) =>(numb1*numb2)</pre>

Solve This Practice Question

You're building a quick greeter for a hotel check-in app. Write an arrow function named **greetGuest** that takes **two parameters, guestName** and **roomNumber**, and returns a string like **"Welcome Alex, your room number is 101"**.

- Store the arrow function in a **const** variable named **greetGuest**
- Call the function with a **name** and **room number of your choice**
- Store the **returned result** in a variable named **greeting**

Immediately Invoked Function Expression (IIFE)

- Normally, a function runs only when it is called.
- An **IIFE runs automatically** when the **JavaScript engine reads it (compiles it)**.
- An IIFE function is **defined** and **executed** at the **same time**.
- **Eg :**

```
// named IIFE
(function hello(){
  console.log("Hello");
})();
```

```
// unnamed IIFE
(function (){
  console.log("Hello there")
})();
```

```
// unnamead Arrow IIFE
(() => {
  console.log("Hello Now");
})();
```

```
// IIFE with parameter
((name) => {
  console.log(`Hello now ${name}`);
})('john')
```

Why is IIFE used ?

- **To create private scope** : Everything inside an IIFE is private to that function.

Eg :

```
(function(){
  let a = 25;
})();
// a is not accesible here
```

- **Avoid polluting global scope** : Variables inside an IIFE cannot be accessed from outside.

Eg :

```
(function () {
  let hidden = 42;
})();
```

```
let result = hidden; // Error: hidden is not defined.
```

- IIFE were heavily used before ES6, so working with older JavaScript code so while working with older code/ legacy code you may encounter IIFE.

Questions for more practise

Question 1 — Restaurant Bill Calculator

You're building a billing feature for a restaurant's website. The app needs to calculate the total bill for a customer based on the items they ordered.

- Declare a regular function named **calculateBill** that takes **two parameters** — **itemPrice** and **quantity** — and a **third parameter discount** that has a **default value of 0**.
- Inside the function, **calculate the subtotal** (**itemPrice * quantity**), then **subtract the discount** from it and **return the final amount**.
- Write a separate **arrow function** named **formatBill** that takes **one parameter amount** and returns a string like **"Your total bill is ₹540"**. Use the **implicit return** (shorthand).
- **Call calculateBill** with an **item price and quantity of your choice**, pass the result into **formatBill**, and store the **final formatted string** in a variable named **receipt**.
- **Call calculateBill again**, this time **without passing a discount**, to **verify the default parameter kicks in**, and **console.log** that result directly.

Question 2 — Website Startup Initialiser

You're building the initialisation logic for a startup landing page. When the JavaScript file loads, the page should immediately set up some config values and display a welcome banner — without polluting the global scope with variables that don't need to be global.

- Declare a let variable named **appName** in the **global scope** and **assign it a name for your app** (e.g. **"LaunchPad"**). This represents a config value the whole app needs access to.
- **Write an IIFE (you can use either the named or arrow syntax) that runs immediately when the script loads.** **Inside it**, declare a let variable named **bannerMessage** that **builds a string using appName** from the global scope, something like **"Welcome to LaunchPad — We're live!"**.
- **Inside the same IIFE**, declare another **let variable named loadTime** and assign it a string like **"Loaded at 09:00 AM"**. This should stay **private to the IIFE**.
- **console.log** both **bannerMessage** and **loadTime** from **within the IIFE**.
- **After the IIFE**, write a **regular named function** called **getAppName** that returns **appName (the global variable)**, and **call and console.log it below**. This shows that global scope is still reachable even after the IIFE runs.
- Finally, try to **console.log(loadTime) outside the IIFE** and **add a comment explaining what happens and why IIFE was the right choice here over a regular function**.

Question 3 — User Profile Card Generator

You're working on a social media app where each user gets a profile card when they sign up. The card generation logic needs to be properly scoped so that private details don't leak outside.

- Create an anonymous function and store it in a variable named **generateCard**. It should take **two parameters** — **username** and **role**.
- **Inside generateCard**, declare a **let variable** named **cardId** and **assign it any number** (this simulates a private internal ID **that shouldn't be accessible outside**).
- Inside **generateCard**, define a **named inner function** called **buildCard** that **uses both username, role, and cardId (from the outer function's scope)** to return a **string** like **"Profile Card | User: alex | Role: Admin | ID: 1042"**.
- Call **buildCard** from within **generateCard** and **return its result**. **Do not call buildCard outside of generateCard**.
- **Store the result** of calling **generateCard** with a **username** and **role of your choice** in a variable named **myCard** and **console.log** it.
- After all this, **try to console.log(cardId) outside the function** and **add a comment explaining what error you'd expect and why**.